

Lab 4 Practice

Objective

In this practice exercise, you will design and implement a Moore finite state machine (FSM) that is controlled by a Linear Feedback Shift Register (LFSR).

You will:

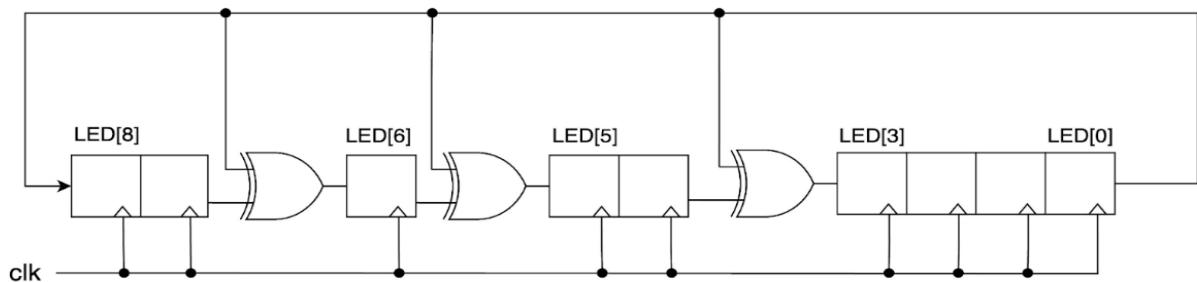
- Implement a 9-bit LFSR in Verilog.
- Use the LFSR output to drive the input of a Moore FSM.
- Build your own FPGA constraints file.

This practice will prepare you for the next lab.

Action Items

a. LFSR

Implement the following 9-bit LFSR (refer to the figure):

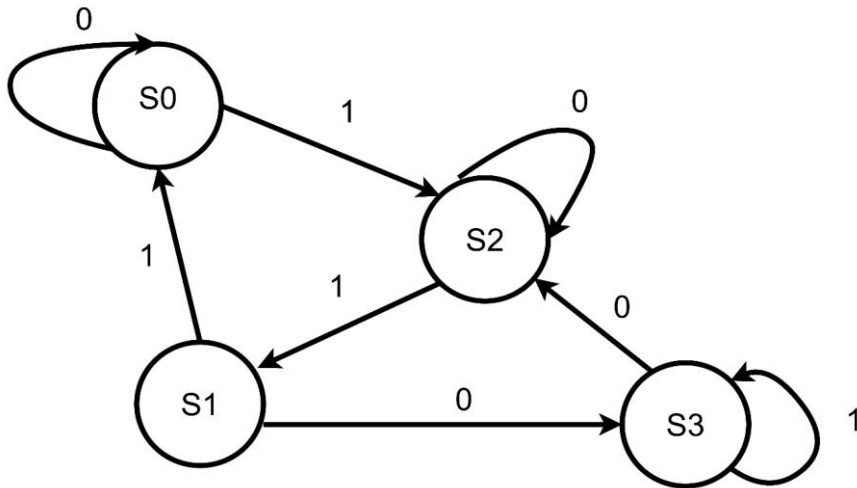


The figure also shows the mapping of the output to the 9 LEDs on the FPGA.

- The LFSR state is displayed on the 9 LEDs (LED[8:0]).
- Reset (rst) initializes the LFSR to **9'b101011001**.
- Button BTNU (shift) shifts the LFSR one cycle to the right.
- The output LED[0] serves as the FSM input bit.

b. FSM Specification

You will design a 4-state **Moore** machine with the following behavior:



- S0 (state encoding: **state[1:0]** to 2'b00)
 - **Reset:** on pressing rst, the FSM always returns to S0.
 - On pressing **next** (BTNR):
 - If the LED[0] is 1, the FSM goes to S2.
 - If the LED[0] is 0, the FSM stays in S0.
- S1 (state encoding: **state[1:0]** to 2'b01)
 - On pressing **next** (BTNR):
 - If the LED[0] is 1, the FSM goes to S0.
 - If the LED[0] is 0, the FSM goes to S3.
- S2 (state encoding: **state[1:0]** to 2'b10)
 - On pressing **next** (BTNR):
 - If the LED[0] is 1, the FSM goes to S1.
 - If the LED[0] is 0, the FSM stay in S2.
- S3 (state encoding: **state[1:0]** to 2'b11)
 - On pressing **next** (BTNR):
 - If the LED[0] is 1, the FSM stays in S3.
 - If the LED[0] is 0, the FSM goes to S2.

c. I/O list

I/O	Connected to	Definition
clk	W5	Clock signal with the frequency of 100MHz
rst	BTNC	Asynchronous active-high reset signal
next	BTNR	FSM control.
shift	BTNU	Shift the LFSR for one cycle
state[1:0]	LED[15:14]	LED15-LED14 control. FSM state indicator.
LED[8:0]	LED[8:0]	LED8-LED0 control. LFSR state display.

Note: All pushbuttons except rst must be debounced and one-pulsed.

Template

You must use the following template for your design:

```
module lab4_practice (  
    input wire clk,  
    input wire rst,  
    input wire next,  
    input wire shift,  
    output reg [1:0] state,  
    output reg [8:0] LED  
);  
    /* Note that output ports can be either reg or wire.  
    * It depends on how you design your module. */  
    // add your design here  
endmodule
```

Attention

- ✓ If you create several modules, merge them into a single Verilog file.
- ✓ This practice prepares you for the next lab. Make sure you fully understand your design.
- ✓ Feel free to ask questions about the specification on the EECLASS forum.
- ✓ DO NOT copy-and-paste code segments from PDFs (hidden characters may cause hard-to-debug syntax errors).